



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“Tarea Práctica #8 Naming/Broadcast (Equipo)”

Sistemas Distribuidos y Paralelos

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 004

Hora: N5

Salón: 4207

Equipo 9

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS
1845517	Andrea Monserrat Méndez Flores	ITS
2109147	Alexia Hernandez Andrade	ITS
2004428	Alejandro Sauqué Pérez	ITS
2077884	Melida Krystal Mendoza Ventura	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 24 de
noviembre de 2024

Contenido

OBJETIVO	3
DIFICULTADES ENCONTRADAS	3
CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)	4
CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)	14
Conclusiones	15

OBJETIVO

Actualizar el proyecto agregando la propiedad de Naming/Broadcast.

DIFICULTADES ENCONTRADAS

Nuestro único y principal problema fue con la conexión entre cliente y servidor usando hamachi, si bien usamos NetworkInterface para un enfoque basado en el descubrimiento de servicios, el cliente no podía encontrar al host correctamente. Cambiamos de puerto y red, pero al final corregimos cliente.

CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)

SERVIDOR

```
1 package Ej;
2
3 import javax.swing.*;
4 import javax.swing.table.DefaultTableModel;
5 import javax.swing.table.TableRowSorter;
6
7 import java.awt.*;
8 import java.io.*;
9 import java.net.*;
10 import java.util.*;
11 import java.util.concurrent.*;
12 import java.util.List;
13
14 import oshi.SystemInfo;
15 import oshi.hardware.CentralProcessor;
16 import oshi.hardware.GlobalMemory;
17 import oshi.hardware.HardwareAbstractionLayer;
18 import oshi.hardware.HWDiskStore;
19 import oshi.hardware.NetworkIF;
20 import oshi.software.os.OperatingSystem;
21
22 public class Server3 {
23     private static final int UPDATE_INTERVAL = 5000; // 5 segundos
24     private static Map<String, ClientHandler> connectedClients = new ConcurrentHashMap<>();
25     private static DefaultTableModel model;
26     private static JTable table;
27     private static final int SERVER_PORT = 3001; // Puerto estático
28
29     private static final String MESSAGE_CODE_SEARCH = "searching-for-server";
30
31
32     //COLORES
33     private static final String RESET = "\033[0m"; //Restablecer el color
34     private static final String RED = "\033[31m"; //R
35     private static final String GREEN = "\033[32m"; //V
36     private static final String YELLOW = "\033[33m"; //A
37     private static final String BLUE = "\033[34m"; //AZ
38
39     public static void main(String[] args) {
40         String[] columnNames = {
41             "ID", "Cliente/Servidor", "Modelo Procesador", "Velocidad (GHz)",
42             "Núcleos", "Disco Duro Total (GB)", "SO", "CPU Libre (%)",
43             "Memoria Libre (GB)", "Ancho Banda Libre (%)", "Disco Libre (GB)", "Estado"
44         };
45
46
47         model = new DefaultTableModel(columnNames, 0) {
48             @Override
49             public boolean isCellEditable(int row, int column) {
50                 return false;
51             }
52         };
53         table = new JTable(model);
54
55         // Configurar el frame
56         JFrame frame = new JFrame("Servidor");
57         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
58         frame.add(new JScrollPane(table));
59         frame.setSize(1200, 400);
60         frame.setVisible(true);
61
62         System.out.println(GREEN + "Esperando conexiones.." + RESET);
63
64         // Se escucha por peticiones de clientes para saber el serverIp (broadcast)
65         Thread ipRequestHandlerThread = new Thread(() -> handleServerIpRequests());
66         ipRequestHandlerThread.start();
67
68         // Configurar el servidor con un puerto estatico
69         ServerSocket serverSocket = null;
70         try {
71             serverSocket = new ServerSocket(SERVER_PORT);
72             // Agregar datos del servidor
73             addServerData();
74         }
```

```

75 // Hilo para actualizar datos del servidor y ordenar filas
76 ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);
77 scheduler.scheduleAtFixedRate(() -> {
78     updateServerData();
79     updateClientsData();
80     sortTableByCpuFree();
81 }, 0, UPDATE_INTERVAL, TimeUnit.MILLISECONDS);
82
83 // Aceptar conexiones de clientes
84 while (true) {
85     Socket clientSocket = serverSocket.accept();
86     ClientHandler handler = new ClientHandler(clientSocket);
87     new Thread(handler).start();
88 }
89 } catch (Exception e) {
90     e.printStackTrace();
91 }
92 }
93
94 private static void handleServerIpRequests() {
95     byte[] buffer = new byte[1024];
96     DatagramSocket socketUDP = null;
97
98     try {
99         System.out.println("Iniciado el servidor UDP");
100         // Creación del socket
101         socketUDP = new DatagramSocket(SERVER_PORT);
102
103         // Siempre atenderá peticiones
104         while (true) {
105             try {
106                 // Preparar el datagrama para recibir la solicitud del cliente
107                 DatagramPacket requestFromClient = new DatagramPacket(buffer, buffer.length);
108
109                 // Recibir el datagrama
110                 socketUDP.receive(requestFromClient);
111                 System.out.println("Recibo la información del cliente");
112
113                 // Convertir el mensaje recibido y mostrarlo
114                 String message = new String(requestFromClient.getData(), 0, requestFromClient.getLength()).trim();
115                 System.out.println("Mensaje del cliente: " + message);
116
117                 // Validar si el mensaje coincide con el código esperado
118                 if (MESSAGE_CODE_SEARCH.equals(message)) {
119                     String serverIp = ServerUtils.getServerIp(); // Obtener la IP del servidor
120                     System.out.println("IP del servidor: " + serverIp);
121
122                     // Obtener el puerto y la dirección del cliente
123                     int clientPort = requestFromClient.getPort();
124                     InetAddress clientAddress = requestFromClient.getAddress();
125
126                     // Preparar la respuesta con la IP del servidor
127                     byte[] responseBuffer = serverIp.getBytes();
128                     DatagramPacket responseForClient = new DatagramPacket(responseBuffer, responseBuffer.length);
129
130                     // Enviar la respuesta
131                     System.out.println("Se envió la IP del servidor al cliente: " + clientAddress);
132                     socketUDP.send(responseForClient);
133                 }
134             } catch (Exception innerException) {
135                 // Manejo de excepciones dentro del ciclo para continuar con otras solicitudes
136                 System.err.println("Error procesando solicitud del cliente: " + innerException.getMessage());
137                 innerException.printStackTrace();
138             }
139         }
140     } catch (Exception e) {
141         // Manejo de excepciones al iniciar el servidor
142         System.err.println("Error iniciando el servidor UDP: " + e.getMessage());
143         e.printStackTrace();
144     } finally {
145         // Asegurarse de cerrar el socket al terminar
146         if (socketUDP != null && !socketUDP.isClosed()) {

```

```

147         socketUDP.close();
148         System.out.println("Socket UDP cerrado.");
149     }
150 }
151 }
152
153
154 private static void addServerData() {
155     SystemInfo si = new SystemInfo();
156     HardwareAbstractionLayer hal = si.getHardware();
157     CentralProcessor processor = hal.getProcessor();
158     OperatingSystem os = si.getOperatingSystem();
159     List<HWDiskStore> diskStores = hal.getDiskStores();
160
161     String processorModel = processor.getProcessorIdentifier().getName();
162     double processorSpeed = processor.getProcessorIdentifier().getVendorFreq() / 1e9;
163     int cores = processor.getLogicalProcessorCount();
164     long diskSize = diskStores.get(0).getSize() / (1024 * 1024 * 1024);
165
166     model.addRow(new Object[] {
167         System.getProperty("user.name"),
168         "SERVIDOR",
169         processorModel,
170         processorSpeed,
171         cores,
172         diskSize,
173         os.toString(),
174         0.0, // CPU Libre
175         0.0, // Memoria Libre
176         0.0, // Ancho de banda
177         0.0, // Disco libre
178         "CONECTADO"
179     });
180 }
181 }
182
183 private static double updateServerData() {
184     SystemInfo si = new SystemInfo();
185     HardwareAbstractionLayer hal = si.getHardware();
186     CentralProcessor processor = hal.getProcessor();
187     GlobalMemory memory = hal.getMemory();
188     List<HWDiskStore> diskStores = hal.getDiskStores();
189
190     // Calcular CPU libre
191     long[] prevTicks = processor.getSystemCpuLoadTicks();
192     try { Thread.sleep(1000); } catch (InterruptedException e) {}
193     double cpuFree = 100.0 - (processor.getSystemCpuLoadBetweenTicks(prevTicks) * 100);
194
195
196
197     // Calcular memoria libre
198     double freeMemory = memory.getAvailable() / (1024.0 * 1024 * 1024);
199
200     // Calcular ancho de banda libre (aproximado basado en la utilización de red)
201     final double bandwidthFree;
202     List<NetworkIF> networks = hal.getNetworkIFs();
203     if (!networks.isEmpty()) {
204         NetworkIF net = networks.get(0);
205         net.updateAttributes();
206         long totalBytes = net.getBytesRecv() + net.getBytesSent();
207         bandwidthFree = 100.0 - (totalBytes % 100); // Simplificado para ejemplo
208     } else {
209         bandwidthFree = 100.0;
210     }
211
212     // Calcular espacio libre en disco
213     File rootDisk = new File("/");
214     double diskFree = rootDisk.getUsableSpace() / (1024.0 * 1024 * 1024); // Convertimos a GB
215
216     // Actualizar fila del servidor
217     SwingUtilities.invokeLater(() -> {
218         model.setValueAt(String.format("%.2f", cpuFree), 0, 7);

```

```

219         model.setValueAt(String.format("%.2f", freeMemory), 0, 8);
220         model.setValueAt(String.format("%.2f", bandwidthFree), 0, 9);
221         model.setValueAt(String.format("%.2f", diskFree), 0, 10);
222     });
223     return cpuFree;
224 }
225
226 private static void updateClientsData() {
227     // Crear una lista temporal de clientes a remover
228     List<String> clientsToRemove = new ArrayList<>();
229
230     // Verificar clientes desconectados
231     connectedClients.forEach((id, handler) -> {
232         if (!handler.isConnected()) {
233             updateClientStatus(id, "DESCONECTADO");
234             clientsToRemove.add(id);
235         }
236     });
237
238     // Remover clientes desconectados
239     clientsToRemove.forEach(connectedClients::remove);
240
241     // Actualizar la tabla solo para clientes conectados
242     for (int i = 0; i < model.getRowCount(); i++) {
243         String clientId = (String) model.getValueAt(i, 0);
244         String status = (String) model.getValueAt(i, 11);
245
246         // Si es el servidor, continuar con el siguiente
247         if (clientId.equals("SERVIDOR")) {
248             continue;
249         }
250
251         // Si el cliente está desconectado, mantener los valores pero en gris
252         if (status.equals("DESCONECTADO")) {
253             table.setRowSelectionInterval(i, i);
254             table.setSelectionBackground(Color.LIGHT_GRAY);
255         }
256     }
257 }
258
259 private static void updateClientStatus(String clientId, String status) {
260     for (int i = 0; i < model.getRowCount(); i++) {
261         if (model.getValueAt(i, 0).equals(clientId)) {
262             model.setValueAt(status, i, 11);
263
264             if (status.equals("DESCONECTADO")) {
265                 // Limpiar todos los campos excepto ID y tipo de cliente
266                 model.setValueAt("", i, 2); // Modelo Procesador
267                 model.setValueAt(0.0, i, 3); // Velocidad
268                 model.setValueAt(0, i, 4); // Núcleos
269                 model.setValueAt(0.0, i, 5); // Disco Duro Total
270                 model.setValueAt("", i, 6); // SO
271                 model.setValueAt(0.0, i, 7); // CPU Libre
272                 model.setValueAt(0.0, i, 8); // Memoria Libre
273                 model.setValueAt(0.0, i, 9); // Ancho Banda Libre
274                 model.setValueAt(0.0, i, 10); // Disco Libre
275
276                 // Cambiar color de fondo a gris
277                 table.setRowSelectionInterval(i, i);
278                 table.setSelectionBackground(Color.LIGHT_GRAY);
279             } else {
280                 table.clearSelection();
281                 table.setSelectionBackground(table.getBackground());
282             }
283             break;
284         }
285     }
286     sortTableByCpuFree();
287 }
288
289 private static void sortTableByCpuFree() {
290     TableRowSorter<DefaultTableModel> sorter = new TableRowSorter<>(model);
291

```

```

292 // Configurar comparador personalizado
293 sorter.setComparator(7, (value1, value2) -> {
294     int row1 = -1, row2 = -1;
295
296     // Encontrar las filas correspondientes a los valores
297     for (int i = 0; i < model.getRowCount(); i++) {
298         if (model.getValueAt(i, 7).equals(value1)) row1 = i;
299         if (model.getValueAt(i, 7).equals(value2)) row2 = i;
300     }
301
302     // Obtener estados de conexión
303     String status1 = model.getValueAt(row1, 11).toString();
304     String status2 = model.getValueAt(row2, 11).toString();
305
306     // Si uno está desconectado y el otro no, el desconectado va al final
307     if (status1.equals("DESCONECTADO") && !status2.equals("DESCONECTADO")) return 1;
308     if (!status1.equals("DESCONECTADO") && status2.equals("DESCONECTADO")) return -1;
309
310     // Si ambos están desconectados, mantener el orden actual
311     if (status1.equals("DESCONECTADO") && status2.equals("DESCONECTADO")) {
312         return row1 - row2;
313     }
314
315     // Asegurarse de que los valores sean válidos para convertir a Double
316     double cpu1 = parseDoubleSafely(value1);
317     double cpu2 = parseDoubleSafely(value2);
318
319     // Ordenar por CPU libre
320     return Double.compare(cpu2, cpu1); // Orden descendente
321 });
322
323 table.setRowSorter(sorter);
324
325 // Aplicar orden descendente en la columna 7 (CPU Libre)
326 List<RowSorter.SortKey> sortKeys = new ArrayList<>();
327 sortKeys.add(new RowSorter.SortKey(7, SortOrder.ASCENDING));
328 sorter.setSortKeys(sortKeys);
329
330 sorter.sort();
331
332 // Ahora, asignamos el tipo "Servidor" a quien tiene el CPU ms alto
333 double maxCpuFree = -1;
334 int maxCpuRow = -1;
335
336 // Encontramos la fila con el CPU ms alto
337 for (int i = 0; i < model.getRowCount(); i++) {
338     double cpuFree = parseDoubleSafely(model.getValueAt(i, 7)); // Valor de CPU Libre en la columna 7
339     if (cpuFree > maxCpuFree) {
340         maxCpuFree = cpuFree;
341         maxCpuRow = i;
342     }
343 }
344
345 // Asignamos "Servidor" a la fila con el CPU ms alto y "Cliente" a las demás
346 for (int i = 0; i < model.getRowCount(); i++) {
347     if (i == maxCpuRow) {
348         model.setValueAt("SERVIDOR", i, 1); // Columna 1 es la que tiene el estado
349     } else {
350         model.setValueAt("CLIENTE", i, 1); // Asignamos "Cliente" a las demás filas
351     }
352 }
353
354 // Método seguro para convertir String a Double
355 private static double parseDoubleSafely(Object value) {
356     try {
357         return Double.parseDouble(value.toString());
358     } catch (NumberFormatException e) {
359         return 0.0; // Si el valor no es un número válido, se devuelve 0.0
360     }
361 }
362
363
364
365 static class ClientHandler implements Runnable {

```



```

366 private final Socket socket;
367 private String clientId;
368 private volatile boolean connected = true;
369 private ObjectInputStream ois;
370
371 public ClientHandler(Socket socket) {
372     this.socket = socket;
373 }
374
375 @Override
376 public void run() {
377     try {
378         ois = new ObjectInputStream(socket.getInputStream());
379         // Esperar hasta que se reciba el ID del cliente
380         Object receivedObject = ois.readObject();
381         if (receivedObject instanceof Map) {
382             Map<String, Object> clientSpecs = (Map<String, Object>) receivedObject;
383             clientId = (String) clientSpecs.get("ID");
384             // Agregar al mapa de clientes conectados
385             connectedClients.put(clientId, this);
386
387             while (connected) {
388                 receivedObject = ois.readObject();
389                 if (receivedObject instanceof Map) {
390                     clientSpecs = (Map<String, Object>) receivedObject;
391                     handleClientData(clientSpecs);
392                 }
393             }
394         }
395         catch (EOFException e) {
396             System.out.println(RED + "Cliente desconectado: " + RESET + clientId);
397             connected = false;
398         }
399         catch (IOException e) {
400             System.out.println(RED + "Error de conexion con el cliente: " + RESET + clientId);
401             connected = false;
402         }
403         catch (ClassNotFoundException e) {
404             e.printStackTrace();
405         }
406         finally {
407             try {
408                 if (ois != null) {
409                     ois.close();
410                 }
411                 socket.close();
412             }
413             catch (IOException e) {
414                 e.printStackTrace();
415             }
416         }
417
418         // Actualizar el estado del cliente en el servidor
419         if (clientId != null) {
420             connected = false;
421             updateClientStatus(clientId, "DESCONECTADO");
422         }
423     }
424 }
425
426 private void handleClientData(final Map<String, Object> clientSpecs) {
427     SwingUtilities.invokeLater(() -> {
428         boolean clientExists = false;
429         String clientId = (String) clientSpecs.get("ID");
430
431         for (int i = 0; i < model.getRowCount(); i++) {
432             if (model.getValueAt(i, 0).equals(clientId)) {
433                 // Si el cliente existe pero estaba desconectado, actualizar su estado
434                 if (model.getValueAt(i, 11).equals("DESCONECTADO")) {
435                     model.setValueAt("CONECTADO", i, 11);
436                     table.clearSelection();
437                     table.setSelectionBackground(table.getBackground());
438                 }
439                 updateRowData(i, clientSpecs);
440                 clientExists = true;
441                 break;
442             }
443         }
444     })
445 }

```

```

440         if (!clientExists) {
441             addNewClient(clientSpecs);
442         }
443
444         // Forzar reordenamiento después de actualizar
445         sortTableByCpuFree();
446     });
447 }
448
449 private void updateRowData(int row, Map<String, Object> specs) {
450     // Solo actualizar si el cliente está conectado
451     if (model.getValueAt(row, 11).equals("CONECTADO")) {
452         model.setValueAt(specs.get("Modelo del Procesador"), row, 2);
453         model.setValueAt(specs.get("Velocidad del Procesador (GHz)"), row, 3);
454         model.setValueAt(specs.get("Nucleos"), row, 4);
455         model.setValueAt(specs.get("Capacidad del Disco Duro (GB)"), row, 5);
456         model.setValueAt(specs.get("Version del Sistema Operativo"), row, 6);
457         model.setValueAt(specs.get("CPU Libre"), row, 7);
458         model.setValueAt(specs.get("Memoria Libre"), row, 8);
459         model.setValueAt(specs.get("Ancho de Banda Libre"), row, 9);
460         model.setValueAt(specs.get("Disco Libre"), row, 10);
461     }
462 }
463
464 private void addNewClient(Map<String, Object> specs) {
465
466     model.addRow(new Object[]{
467         specs.get("ID"),
468         "CLIENTE",
469         specs.get("Modelo del Procesador"),
470         specs.get("Velocidad del Procesador (GHz)"),
471         specs.get("Nucleos"),
472         specs.get("Capacidad del Disco Duro (GB)"),
473         specs.get("Version del Sistema Operativo"),
474         specs.get("CPU Libre"),
475         specs.get("Memoria Libre"),
476         specs.get("Ancho de Banda Libre"),
477         specs.get("Disco Libre"),
478         "CONECTADO"
479     });
480 }
481
482 public boolean isConnected() {
483     return connected;
484 }
485 }
486 }

```

CLIENTE

```

1 package Ej;
2
3 import oshi.SystemInfo;
4 import oshi.hardware.*;
5 import oshi.software.os.OperatingSystem;
6
7 import java.io.*;
8 import java.net.DatagramPacket;
9 import java.net.DatagramSocket;
10 import java.net.InetAddress;
11 import java.net.Socket;
12 import java.util.HashMap;
13 import java.util.List;
14 import java.util.Map;
15 import java.util.Scanner;
16
17 public class Client3 {
18     private static final int UPDATE_INTERVAL = 5000; // 5 seg
19     private static final int SERVER_PORT = 3001;
20
21     private static final String MESSAGE_CODE_SEARCH = "searching-for-server";
22
23     // COLORES
24     private static final String RESET = "\033[0m"; // Restablecer el color
25     private static final String RED = "\033[31m"; // Rojo
26     private static final String GREEN = "\033[32m"; // Verde
27     private static final String YELLOW = "\033[33m"; // Amarillo
28     private static final String BLUE = "\033[34m"; // Azul
29
30     public static void main(String[] args) {
31         byte[] buffer = new byte[1024];
32         Scanner scanner = new Scanner(System.in);
33         DatagramSocket socketUDP = null;
34
35         try {
36             String serverIp = null;
37
38             // Crear el socket UDP
39             socketUDP = new DatagramSocket();
40
41             // Enviar solicitud de búsqueda al servidor
42             InetAddress serverAddress = InetAddress.getByName("255.255.255.255");
43             String messageCode = MESSAGE_CODE_SEARCH;
44             buffer = messageCode.getBytes();
45
46             System.out.println("Server Address: " + serverAddress);
47             DatagramPacket serverSearchRequest = new DatagramPacket(buffer, buffer.length, serverAddress, SERVER_PORT);
48             System.out.println("Enviando datagrama de búsqueda del servidor...");
49             socketUDP.send(serverSearchRequest);
50
51             // Esperar respuesta del servidor
52             DatagramPacket responseDatagram = new DatagramPacket(buffer, buffer.length);
53
54             try {
55                 socketUDP.setSoTimeout(20000); // Esperar 5 segundos
56                 socketUDP.receive(responseDatagram);
57                 System.out.println("Respuesta recibida del servidor.");
58
59                 // Procesar la respuesta
60                 String response = new String(responseDatagram.getData()).trim();
61                 String[] parts = response.split("-server");
62                 // La IP estará en la primera parte
63                 String ip = parts[0];
64                 if (!ip.matches("\\d+\\.\\d+\\.\\d+\\.\\d+")) {
65                     throw new IllegalArgumentException("Respuesta inválida: " + response);
66                 }
67

```

```

68         serverIp = ip;
69         System.out.println("Dirección IP del servidor: " + serverIp);
70     } catch (IOException e) {
71         System.out.println(e);
72         System.out.println(RED + "No se encontró un servidor disponible en la red." + RESET);
73         System.out.println(YELLOW + "TODO: Implementar funcionalidad para que este cliente actúe como ser
74         return; // Salir del programa hasta que se implemente el TODO
75     }
76
77     // Cerrar el socket UDP ya que no será necesario
78     socketUDP.close();
79
80     // =====
81
82     // Conectar al servidor
83     int port = SERVER_PORT;
84     String clientId = getUserId();
85     Socket socket = new Socket(serverIp, port);
86     ObjectOutputStream oos = new ObjectOutputStream(socket.getOutputStream());
87
88     // Enviar actualizaciones periódicas
89     Thread updateThread = new Thread() -> {
90         try {
91             while (!Thread.interrupted()) {
92                 Map<String, Object> specs = getSystemSpecs(clientId);
93                 oos.writeObject(specs);
94                 oos.reset(); // Limpiar cache
95                 Thread.sleep(UPDATE_INTERVAL);
96             }
97         } catch (Exception e) {
98             System.out.println(RED + "Conexión perdida con el servidor." + RESET);
99         }
100     });
101
102     updateThread.start();
103
104     // Esperar ENTER para desconectarse
105     System.out.println(RED + "\nPresiona ENTER para desconectarte." + RESET);
106     scanner.nextLine();
107     updateThread.interrupt();
108     socket.close();
109
110     } catch (Exception e) {
111         System.out.println(RED + "Ocurrió un error inesperado: " + e.getMessage() + RESET);
112         e.printStackTrace();
113     } finally {
114         if (socketUDP != null && !socketUDP.isClosed()) {
115             socketUDP.close();
116         }
117         scanner.close();
118     }
119 }
120
121 private static String getUserId() {
122     return System.getProperty("user.name");
123 }
124
125 private static Map<String, Object> getSystemSpecs(String clientId) {
126     SystemInfo si = new SystemInfo();
127     HardwareAbstractionLayer hal = si.getHardware();
128     CentralProcessor processor = hal.getProcessor();
129     GlobalMemory memory = hal.getMemory();
130     List<HWDiskStore> diskStores = hal.getDiskStores();
131     List<NetworkIF> networks = hal.getNetworkIFs();
132     OperatingSystem os = si.getOperatingSystem();
133
134     long[] prevTicks = processor.getSystemCpuLoadTicks();
135     try { Thread.sleep(1000); } catch (InterruptedException e) { }
136     double cpuFree = 100.0 - (processor.getSystemCpuLoadBetweenTicks(prevTicks) * 100);

```

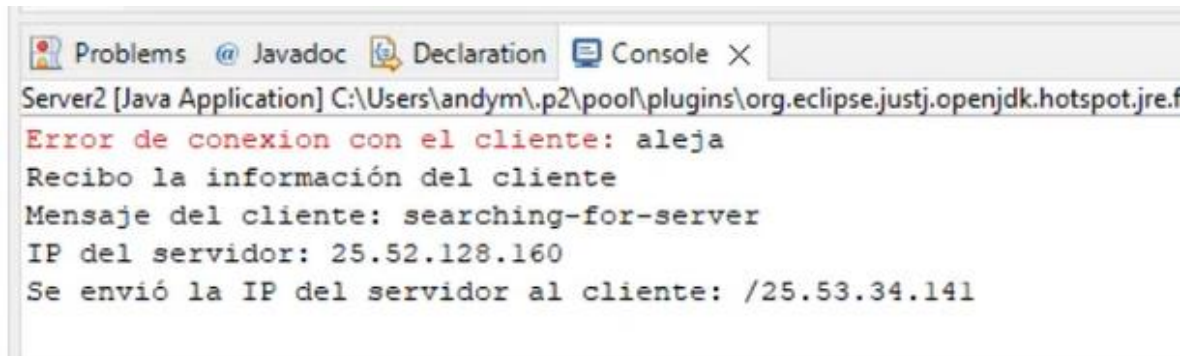
```

136     double cpuFree = 100.0 - (processor.getSystemCpuLoadBetweenTicks(prevTicks) * 100);
137
138     double freeMemory = memory.getAvailable() / (1024.0 * 1024 * 1024);
139
140     double bandwidthFree = 100.0;
141     if (!networks.isEmpty()) {
142         NetworkIF net = networks.get(0);
143         net.updateAttributes();
144         long totalBytes = net.getBytesRecv() + net.getBytesSent();
145         bandwidthFree = 100.0 - (totalBytes % 100);
146     }
147
148     File rootDisk = new File("/");
149     double diskFree = rootDisk.getUsableSpace() / (1024.0 * 1024 * 1024);
150
151     Map<String, Object> specs = new HashMap<>();
152     specs.put("ID", clientId);
153     specs.put("Modelo del Procesador", processor.getProcessorIdentifier().getName());
154     specs.put("Velocidad del Procesador (GHz)", processor.getProcessorIdentifier().getVendorFreq() / 1e9);
155     specs.put("Nucleos", processor.getLogicalProcessorCount());
156     specs.put("Capacidad del Disco Duro (GB)", diskStores.get(0).getSize() / (1024 * 1024 * 1024));
157     specs.put("Version del Sistema Operativo", os.toString());
158     specs.put("CPU Libre", String.format("%.2f", cpuFree));
159     specs.put("Memoria Libre", String.format("%.2f", freeMemory));
160     specs.put("Ancho de Banda Libre", String.format("%.2f", bandwidthFree));
161     specs.put("Disco Libre", String.format("%.2f", diskFree));
162
163     System.out.println(GREEN + "\n----- Datos Actualizados -----" + RESET);
164     System.out.println("ID: " + specs.get("ID"));
165     System.out.println(BLUE + "Modelo del Procesador: " + specs.get("Modelo del Procesador"));
166     System.out.println("Velocidad del Procesador: " + specs.get("Velocidad del Procesador (GHz)") + " GHz");
167     System.out.println("Nucleos: " + specs.get("Nucleos"));
168     System.out.println("Capacidad del Disco Duro: " + specs.get("Capacidad del Disco Duro (GB)") + " GB");
169     System.out.println("Versión del Sistema Operativo: " + specs.get("Version del Sistema Operativo"));
170     System.out.println(YELLOW + "CPU Libre: " + specs.get("CPU Libre") + "%");
171
172     System.out.println("Memoria Libre: " + specs.get("Memoria Libre") + " GB");
173     System.out.println("Ancho de Banda Libre: " + specs.get("Ancho de Banda Libre") + "%");
174     System.out.println("Disco Libre: " + specs.get("Disco Libre") + " GB");
175
176     return specs;
177 }

```

CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)

SERVIDOR



The screenshot shows the Eclipse IDE's Console window. The title bar includes tabs for 'Problems', 'Javadoc', 'Declaration', and 'Console'. The console output for 'Server2 [Java Application]' shows the following sequence of events:

```
Server2 [Java Application] C:\Users\andym\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.f
Error de conexion con el cliente: aleja
Recibo la información del cliente
Mensaje del cliente: searching-for-server
IP del servidor: 25.52.128.160
Se envió la IP del servidor al cliente: /25.53.34.141
```

CLIENTE

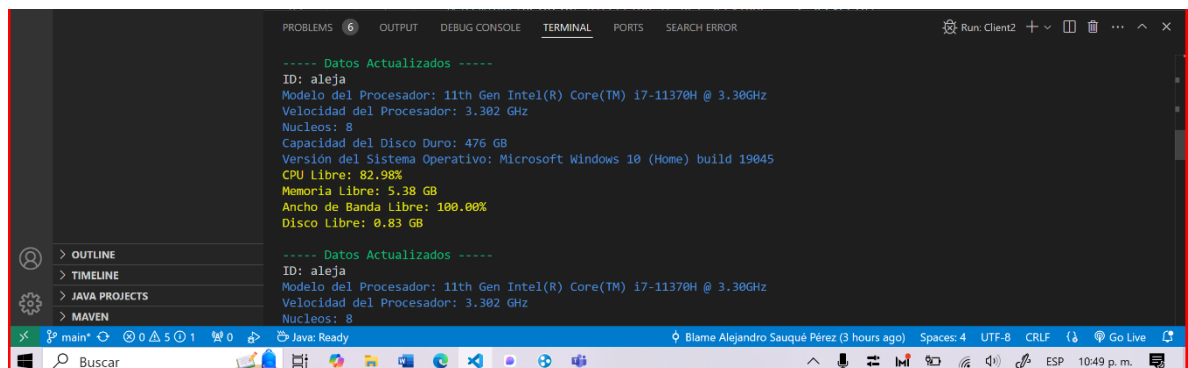


The screenshot shows the Eclipse IDE's Console window for the client application. The output includes the following information:

```
Server Address: /255.255.255.255
Enviando datagrama de búsqueda del servidor...
Respuesta recibida del servidor.
Dirección IP del servidor: 25.52.128.160

Presiona ENTER para desconectarte.
22:45:12.283 [Thread-0] WARN oshi.util.platform.windows.PerfCounterWildcardQuery - Disabling further attempts to query PhysicalDisk.
22:45:17.916 [Thread-0] DEBUG oshi.software.os.windows.WindowsOperatingSystem - Debug privileges not enabled.
22:45:17.922 [Thread-0] WARN oshi.util.platform.windows.PerfCounterWildcardQuery - Disabling further attempts to query Processor Information.

----- Datos Actualizados -----
ID: heidi
Modelo del Procesador: 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz
Velocidad del Procesador: 2.688 GHz
Núcleos: 12
Capacidad del Disco Duro: 476 GB
Versión del Sistema Operativo: Microsoft Windows 11 (Home) build 22631
CPU Libre: 97.41%
Memoria Libre: 3.94 GB
Ancho de Banda Libre: 100.00%
Disco Libre: 56.07 GB
```



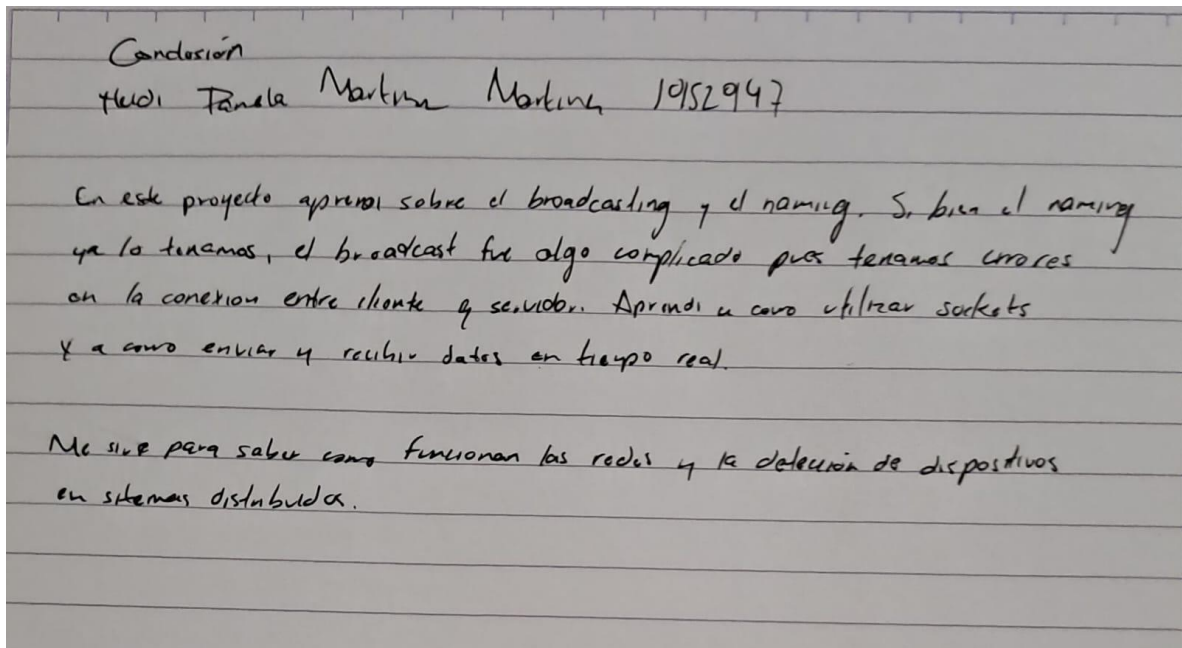
This screenshot shows the Eclipse IDE's Console window with the 'TERMINAL' tab selected. It displays system information for the client application, including CPU, memory, and disk usage. The output is as follows:

```
----- Datos Actualizados -----
ID: aleja
Modelo del Procesador: 11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz
Velocidad del Procesador: 3.302 GHz
Núcleos: 8
Capacidad del Disco Duro: 476 GB
Versión del Sistema Operativo: Microsoft Windows 10 (Home) build 19H45
CPU Libre: 82.98%
Memoria Libre: 5.38 GB
Ancho de Banda Libre: 100.00%
Disco Libre: 0.83 GB

----- Datos Actualizados -----
ID: aleja
Modelo del Procesador: 11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz
Velocidad del Procesador: 3.302 GHz
Núcleos: 8
```

Conclusiones

1952947 - Heidi Pamela Martínez Martínez



Andrea Monserrat Méndez Flores

Conclusión:

Esta tarea fue bastante compleja para el equipo, esto debido a que no lográbamos conectarnos al servidor, y después surgieron más problemas ya que al lograr conectarnos, aún así no lográbamos mostrar la información en la tabla, una vez que se logró conectar lo pudimos observar por medio de la consola y así saber que si estaba funcionando, pero no logramos que apareciera en pantalla la tabla con los datos.

Conclusión Tarea práctica #8

Alexia Hernandez Andrade / 2109147

En esta actividad se logró implementar la funcionalidad de Namig/Broadcast en nuestro sistema distribuido, lo que representa un avance significativo en la comunicación y coordinación entre dispositivos. A través de esta mejora el sistema ahora permite que los dispositivos conectados puedan identificar y localizar a otros participante en la red de manera eficiente, simplificando interacción entre servidores y clientes.

Alejandro Sauqué Pérez

Esta práctica me pareció bastante enriquecedora debido a que nos hace pensar no solo en los conceptos teóricos de un verdadero sistema distribuido sino que tenemos que cuidar los detalles de cómo se integran técnicas como UDP en el sistema que hemos ido construyendo. Me parece que la parte desafiante es hacer que todo funcione desde Hamachi a distancia.

